SENIOR QA ASSESSMENT

Assessment Report

OrangeHRM UI Automation & Manual Testing – what was delivered and how it was verified

Candidate	Moomen Abdelrazik
Date	15 October 2026
Application under test	https://opensource-demo.orangehrmlive.com/ (OrangeHRM OS 5.9)
Automation stack	Java 17 · Maven 3.9 · Selenium 4.25 · TestNG 7.10 · Allure 2
Execution result	**16 / 16 tests passed** (full suite, 4 min 13 s)

1. Executive summary

16 / 16	10 / 10	6	64
automated tests passed	functional steps automated	negative scenarios (2 required)	manual test cases designed

The assessment has two parts, and both are complete:

- **Automation:** a Java/Maven framework (Selenium WebDriver, TestNG, Allure, Page Object Model) that automates all ten functional steps on the OrangeHRM demo, with configuration outside the code, generated test data, automatic cleanup, checkpoint and failure screenshots, and an execution report.
- **Manual testing:** a review of the *Employee Leave Request* story (questions, missing requirements, assumptions, risks), a test design with 64 test cases plus state-transition, decision-table and exploratory charters, and a full bug report for the 3-days-shown / 4-days-deducted defect.

All deliverables are in the Git repository: framework, README, automated tests, screenshots and example reports, plus the manual testing documents.

2. Requirement verification

Each requirement from the assessment was checked against the code and against an actual run on the live demo.

2.1 Functional scope (automation)

Step	Requirement	How it is covered	Evidence	Status
1	Login page displayed, username / password / login button available	`LoginTest.loginPageDisplaysAllControls` (soft assertions on each control)	Test passed	**Done**
1	Login succeeds, dashboard and user info displayed	`LoginTest.validLoginOpensDashboard` – URL, header, user name, avatar, widgets	Screenshot 01	**Done**
2	Search existing employee by name, validate results, no hard-coded row	`EmployeeSearchTest` – target taken from the live list, rows read by column header and found by id	Screenshot 03	**Done**
3	Create employee with generated data; success message; Employee ID; found in list	`EmployeeTest.createEmployee` – `Automation Test Qa<random>`, toast "Successfully Saved", generated id captured, list search by id	Screenshot 02	**Done**
4	Re-open the created employee and prove it was persisted	`EmployeeTest.validateCreatedEmployee` – new browser session, search by name, open record, compare every field and id	Test passed	**Done**
5	Edit fields, success message, updated value visible	`EmployeeTest.editEmployee` – new last name and Other Id, toast "Successfully Updated"	Screenshot 04	**Done**

Step	Requirement	How it is covered	Evidence	Status
6	Persistence after refresh and after navigating away	`EmployeeTest.updatedValuesPersistAfterRefreshAndNavigation`	Test passed	**Done**
7	At least two filters, data validated, reset restores result set	`EmployeeFilterTest` – Job Title + Employment Status, and Name + Employee Id; every row checked; count compared after Reset	Screenshot 07	**Done**
8	Leave request with available type and calculated dates; message; located afterwards	`LeaveTest.submitLeaveRequest` – type with balance, random future working day, found in My Leave as *Pending Approval*, then cancelled	Screenshot 06	**Done**
9	At least two negative scenarios	`NegativeValidationTest` (5) + `LoginTest.invalidPasswordIsRejected` – each also proves nothing was created / changed	Screenshots 05	**Done (6)**
10	Test-data cleanup considered and explained	`@AfterSuite` deletes created employees; leave cancelled in `finally`; README section "Cleanup"	Run log: "Deleted employee ..."	**Done**

2.2 Framework and non-functional requirements

Requirement	Implementation	Status
Credentials not hard-coded in tests, stored in configuration	`config.properties`, overridable by `-D` and environment variables (`ORANGEHRM_USERNAME` / `ORANGEHRM_PASSWORD`)	**Done**
No hard-coded row numbers	Tables read into objects by column header; rows located by id / name / comment	**Done**
Test data not permanently hard-coded	`TestDataFactory`: unique names, ids, comments; leave dates calculated at runtime; date format read from the application	**Done**
Screenshots at checkpoints (login, creation, search, update, negative, leave)	`ScreenshotUtils.checkpoint` – 9 screenshots committed in `docs/screenshots`	**Done**
Automatic screenshot on failure	`TestListener.afterInvocation` captures screenshot + page source while the browser is open	**Done – verified with a deliberate failure**
Report with name, status, duration, failure information, failure evidence	Allure: `docs/report/index.html` (green run) and `docs/report-with-failure/index.html` (includes a failing test with screenshot and page source)	**Done**
Logical organisation: smoke / regression / negative	TestNG groups on every test	**Done**
Run individual test: `mvn test -Dtest=EmployeeTest`	Surefire `-Dtest`	**Verified**
Run suite: `mvn test -DsuiteXmlFile=testng.xml`	Maven profile activated by the property	**Verified (16 passed)**
Run group: `mvn test -Dgroups=smoke`	TestNG groups via Surefire	**Verified (4 passed)**
README with prerequisites, versions, technologies, architecture, structure, execution, reports, configuration, test-data strategy	`README.md`, sections 1–11	**Done**

Requirement	Implementation	Status
4–6 screenshots	9 provided (8 checkpoints + 1 failure example)	**Done**
Example execution report	Two single-file Allure reports in `docs/`	**Done**
Manual testing: review, test design, bug report	`docs/manual-testing/01-03` and the separate *Manual Testing* PDF	**Done**

3. Automation framework

3.1 Architecture

```
tests  (LoginTest, EmployeeSearchTest, EmployeeTest, EmployeeFilterTest, LeaveTest, NegativeValidationTest)
  ├── extend  BaseTest      browser per test, login, @AfterSuite cleanup
  │             ├── DriverManager / DriverFactory   ThreadLocal WebDriver, Selenium Manager
  │             ├── TestListener                     failure screenshot + page source, logs, Allure
environment
  │             └── ConfigReader                     properties -> environment variables -> -D overrides
  ├── use     flows         reusable business flows (create / open employee, ensure leave balance)
  └── use     pages         Page Objects: BasePage + 8 screens (explicit waits, selects, autocompletes,
toasts, tables)
utils: TestDataFactory · DateUtils · ScreenshotUtils · TestDataRegistry      models: Employee · EmployeeRow ·
LeaveRow
```

Design choices worth highlighting:

- **A fresh browser for every test.** Each check reads data back from the server rather than from the page state left by a previous step, which is what makes the persistence checks meaningful.
- **Explicit waits only.** No `Thread.sleep` and no implicit waits. OrangeHRM-specific behaviour (loading spinners, toasts, the Vue-bound inputs, the date picker) is handled once in `BasePage`.
- **Independent of the shared data.** The demo is public and changes constantly, so search and filter targets are chosen from live data and the tests only assert on data they created or read at runtime.
- **Ready for parallel runs.** The driver is held per thread, so classes can run in parallel by changing one attribute in the suite file.

3.2 Execution results (full suite, `mvn clean test -DsuiteXmlFile=testng.xml`)

#	Test	Group(s)	Duration	Result
1	LoginTest.loginPageDisplaysAllControls	smoke, regression	2.1 s	**Passed**
2	LoginTest.validLoginOpensDashboard	smoke, regression	6.1 s	**Passed**
3	LoginTest.invalidPasswordIsRejected	negative, regression	3.7 s	**Passed**
4	EmployeeSearchTest.searchExistingEmployeeByName	smoke, regression	9.7 s	**Passed**
5	EmployeeTest.createEmployee	smoke, regression	11.3 s	**Passed**
6	EmployeeTest.validateCreatedEmployee	regression	5.3 s	**Passed**
7	EmployeeTest.editEmployee	regression	5.1 s	**Passed**
8	EmployeeTest.updatedValuesPersistAfterRefreshAndNavigation	regression	11.9 s	**Passed**
9	EmployeeFilterTest.filterByEmployeeNameAndId	regression	10.0 s	**Passed**
10	EmployeeFilterTest.filterByJobTitleAndEmploymentStatus	regression	9.6 s	**Passed**
11	LeaveTest.submitLeaveRequest	regression	22.5 s	**Passed**

#	Test	Group(s)	Duration	Result
12	NegativeValidationTest.createEmployeeWithoutRequiredFields	negative, regression	7.2 s	**Passed**
13	NegativeValidationTest.createEmployeeWithTooLongEmployeeId	negative, regression	5.2 s	**Passed**
14	NegativeValidationTest.updateEmployeeWithEmptyFirstName	negative, regression	12.5 s	**Passed**
15	NegativeValidationTest.leaveRequestWithToDateBeforeFromDate	negative, regression	20.0 s	**Passed**
16	NegativeValidationTest.leaveRequestWithoutLeaveType	negative, regression	20.8 s	**Passed**

Total suite time: **252.9 s**, including the browser start for every test and the cleanup at the end. The smoke group (`-Dgroups=smoke`) runs 4 tests in about 54 s.

3.3 Negative scenarios

Scenario	Validation shown	Proof that nothing was saved
Add employee without first / last name	`Required` under both fields	Still on Add Employee; search by the proposed id returns no record
Employee Id longer than 10 characters	`Should not exceed 10 characters`	Search by the generated last name returns 0 records
Update employee with empty first name	`Required`	After reload the original first name and header are unchanged
Leave with To Date before From Date	`To date should be after from date`	No success message, still on Apply Leave
Leave without leave type	`Required`	No success message, still on Apply Leave
Login with wrong password	`Invalid credentials`	Still on the login page, dashboard not reachable

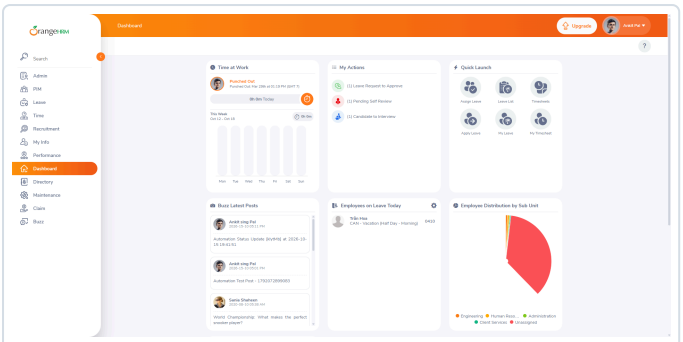
3.4 Test data and cleanup

- **Unique data:** employees are created as `Automation Test Qa<8 random letters>`, and the Employee Id generated by OrangeHRM becomes the key used in later steps. Edited values and leave comments are generated too.
- **Runtime dates:** each leave request uses a random working day 7–45 days ahead. If that day overlaps an existing request, another day is tried. The date format (`yyyy-dd-mm` on the demo) is read from the field placeholder.
- **Data set-up:** if the logged-in user has no leave balance, an entitlement is added through *Leave → Entitlements*.
- **Cleanup:** every created employee is registered and deleted in `@AfterSuite(alwaysRun = true)`, even when tests fail. The leave request is cancelled in a `finally` block, since leave cannot be deleted through the UI and cancelling restores the balance. Cleanup can be switched off with `-Dcleanup.enabled=false` when debugging.

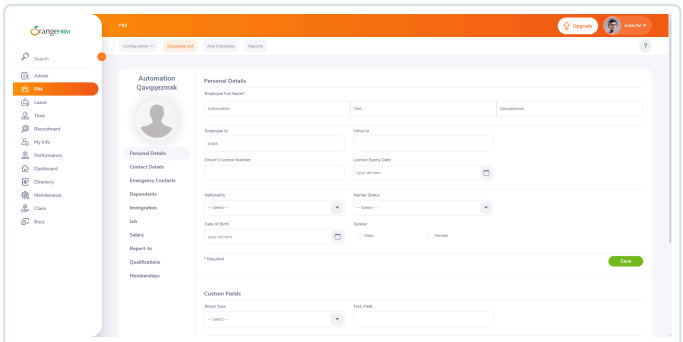
3.5 Observations found while automating

- **Profile header not refreshed:** after saving *Personal Details*, the name in the header keeps its old value until the page is reloaded, although the form fields update. I would report this as a minor UI defect. The tests check the header after a refresh or navigation instead.
- **Unusual date format:** the demo uses `yyyy-dd-mm`. Hard-coding a format would break the tests if the setting changed, so the framework reads it at runtime.
- **Windows environment notes:** HTTPS inspection on the network and JDK 17's internal socket can block Maven or browser start-up. The README has a short troubleshooting section for both.

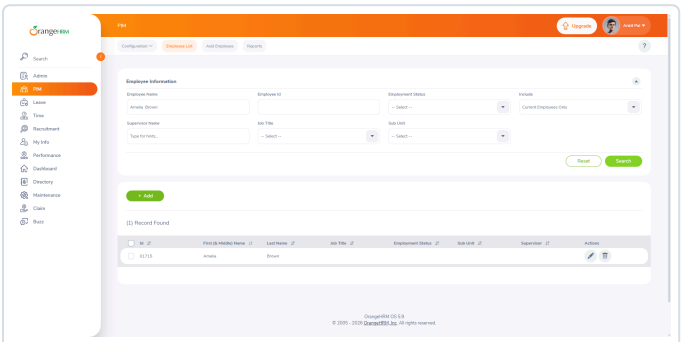
4. Screenshots



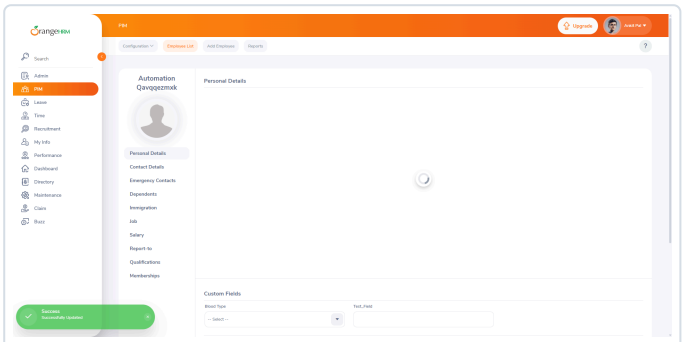
01 – Successful login / dashboard



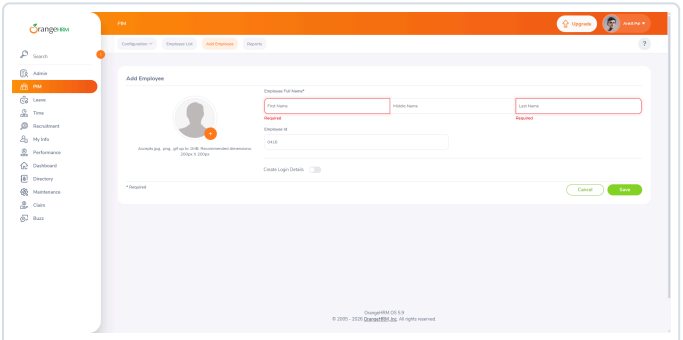
02 – Employee created with generated name and Employee Id



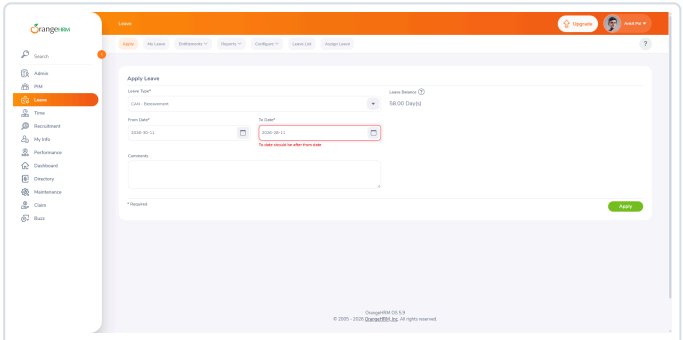
03 – Employee search result



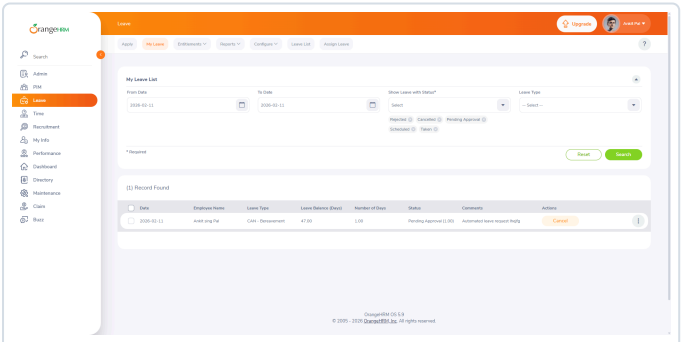
04 – Updated employee



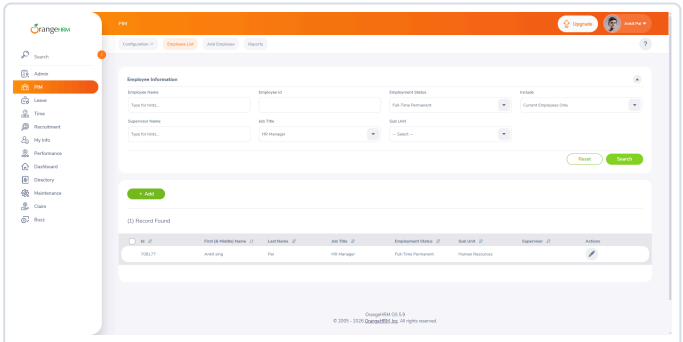
05 – Negative: required field validation



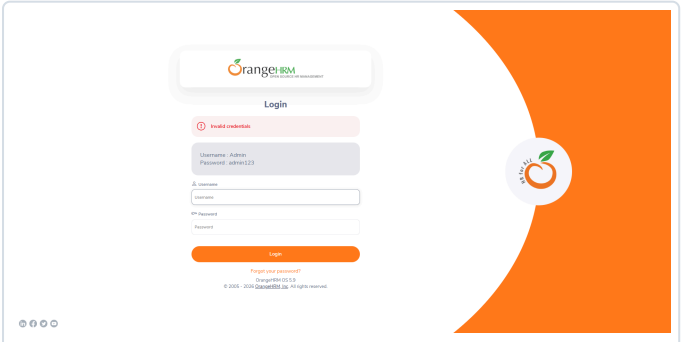
05 – Negative: invalid leave dates



06 – Leave request in My Leave (Pending Approval)



07 – Filter by Job Title + Employment Status



08 – Screenshot captured automatically on a (deliberate) failure

5. Manual testing – what was done

The full manual deliverables are in the separate document **"Manual Testing – Employee Leave Request"** and in `docs/manual-testing`. This section summarises them.

5.1 Requirements review

- **Questions per acceptance criterion:** all 10 acceptance criteria were reviewed. The main gaps are:
  - how leave days are calculated (weekends, holidays, half days);
  - when the balance is deducted (at submission or at approval);
  - what happens when the balance is insufficient;
  - which requests can be cancelled;
  - the undefined term "invalid leave requests" in AC10.
- **Missing requirements:** 8 groups were identified:
  - validation rules;
  - balance rules;
  - the full status model (Cancelled, Taken, partial approval);
  - manager decision details;
  - notifications;
  - roles and permissions;
  - audit trail;
  - non-functional needs.
- **Business-flow review:** the flow has no branch for cancellation, insufficient balance or overlapping requests, and no escalation when the manager does not decide.
- **Assumptions and risks:** 8 assumptions to confirm, and 8 risks rated by likelihood and impact. The highest risks are mismatched day calculation, time-zone date shifts and a balance that is not restored.

5.2 Test design

Area	Content
Techniques	Equivalence partitioning, boundary values, decision table, state transition, role-based testing, error guessing, exploratory charters
Scenarios	12 scenarios mapped to the acceptance criteria (traceability table)
Test cases	64 test cases with preconditions, data, expected result and priority (P1–P3)
Day calculation	Every case checks that the days displayed, the days stored and the balance change all agree
Validation	Decision table with 8 rules: missing type or date, To before From, weekend only, insufficient balance, overlap, invalid date, outside the leave period
Status flow	Pending → Scheduled → Taken, Pending → Rejected, Pending / Scheduled → Cancelled, plus invalid transitions
Security	Employee sees and cancels only their own requests, including at API level
Exploratory	4 charters: calculation edge cases, concurrency, balance life cycle, configuration changes

5.3 Bug report – LEAVE-001

> **Leave balance is reduced by 4 days for a 3-day leave request (10–12 Aug).** Severity Major, priority High. The report contains preconditions (balance 10, Mon–Fri work week, no holiday), exact steps, expected (3 days, balance 7)

> vs actual (3 days shown, balance 6), the evidence to attach (screenshots, network capture, database rows, time zones), the business impact, and hypotheses to investigate: off-by-one on the end date, time-zone shift, different calendars for display and deduction, partial-day handling, double deduction, and stale cache.

6. How to run

```
mvn clean test                              # all tests
mvn test -Dtest=EmployeeTest                # one class
mvn clean test -DsuiteXmlFile=testng.xml    # TestNG suite
mvn clean test -Dgroups=smoke               # group (smoke | regression | negative)
mvn allure:serve                            # open the Allure report
```

On PowerShell, put quotes around each `-D` argument, for example `mvn test "-Dgroups=smoke"`.